UNIVERSITY OF TÜBINGEN

Design and Implementation of an RNA-seq Pipeline

BIOINF4270: Computational Workflows for Biomedical Data

Nina Röhre, Laura Schüle
Student ID: 6939386, 7043581

Professor: Prof. Dr. rer. nat. Sven Nahnsen

24 October 2026

Table of Contents

1 Introduction 1
2 Methodology 2
2.1 Nf-core pipeline Structure 2
2.2 Implementation 2
3 Results 3
3.1 Pipeline implementation and execution 3
3.2 Test sets and profiles 3
3.3 Processing steps and generated outputs 4
3.4 Workflow performance and parallelization 5
4 Discussion 7
4.1 Evaluating the pipeline according to FAIR principles 7
4.2 Outlook for future improvements 8
4.3 Conclusion 9
References 10
A Appendix 13

List of Figures

Figure 1 Metromap of the pipeline picturing main steps. Red branch: Alignment with HISAT2. Blue dashed branch: Pseudoalignment and Quantification with Salmon . 3
Figure 2 Execution timeline generated by Nextflow for the test profile with duplicate sample names.Colours indicate tasks belonging to the same process and the actual execution time of the task. Gray section before the colored section indicates time between task submission and the start of the execution. Gray sections after the colored section indicates time needed to finalize the task after its main execution, for example clean up and other processing overhead. 6
Figure 3 Execution timeline generated by Nextflow for the test profile without duplicate sample names.Colours indicate tasks belonging to the same process and the actual execution time of the task. Gray section before the colored section indicates time between task submission and the start of the execution. Gray sections after the colored section indicates time needed to finalize the task after its main execution, for example clean up and other processing overhead. 7

List of Tables

Table 1 Sample sheet for test profile “test” including duplicate sample names and single and paired-end sequencing data. 4
Table 2 Sample sheet for test profile “test_without_duplicates” including single and paired-end sequencing data. 4
Table 3 Workflow components and their software versions. Note that MAKE_TRANSCRIPTS_FASTA reports STAR version, even when STAR is not used. 13
Table 4 nf-core modules and subworkflows used in the RNA-seq analysis and their respective functions. 14
Table 5 FAIR Principles for Research Software (FAIR4RS). Defined by Barker et al. (2022) (Barker et al., 2022). 15

1 Introduction

A pipeline in its traditional sense is a system of connected tubes transporting a liquid or a gas. Analogous to this, a bioinformatics pipeline transports data through various transformations, usually done by a third-party executable command line software. Bioinformatics pipelines find applicability across many subdisciplines and numerous types of frameworks have been developed (Leipzig, 2017).

One of the many challenges for bioinformatics research is to ensure reproducibility of computational analysis. Reporting all computational steps in enough detail to ensure reproducibility within a research paper would be infeasible. Not just due to the extent of description necessary, but also due to differences in hardware, operating systems and software versions, as well as the lack of shared rules and principles. To address the broader issue of good data management, a set of FAIR principles were proposed. These principles were developed based on four primary principles: Findability, Accessibility, Interoperability, and Reusability, with special emphasis put upon enabling machines and individuals to find and use data. (Sansone et al., 2019; Wilkinson et al., 2016)

Nf-core is a community project that leverages the FAIR principles to improve the reproducibility of pipelines. The nf-core community develops and collects collaborative, peer-reviewed, best-practice pipelines that were built with Nextflow. Nextflow is a dataflow and workflow programming system. Due to the use of Nextflow, these pipelines can run on most computational infrastructure and support container technologies such as docker and singularity. Nf-core also offers the use of community build tools for automated pipeline creation, testing, deployments, and synchronization. (Ewels et al., 2020)

RNA sequencing (RNA-seq), first introduced in 2008, is a technique with the goal to identify differentially expressed and co-regulated genes, and to infer their biological meaning. RNA-seq consists of multiple steps such as: 1. Conversion of RNA into cDNA, 2. Fragmentation, 3. Adapter ligation, 4. Index ligation, 5. High-throughput sequencing of cDNA fragments, 6. Demultiplexing. The High-throughput data then has to be analyzed using a bioinformatics pipeline. While typically such a pipeline consists of pre-processing, genome alignment & quantification, quality control and post-processing steps, there is a lack of consensus on the standardized RNA-seq processing workflow. This lack of consensus can hinder the reproducibility and comparability of research finding. (Koch et al., 2018)

The nf-core community developed an extensive RNA-seq analysis pipeline. This pipeline makes use of multiple bioinformatics tools to process and analyze RNA sequencing data. The pipeline combines tools for quality control, read alignment and quantification, and downstream analysis to produce summarized gene expression data and quality control reports. (nf-core/rnaseq, n.d.)

Within the scope of the course "Computational workflows for biomedical data" this project aimed to develope its own RNA-seq nf-core pipeline. This report describes the development and evaluation of this pipeline. The methodology(Section 2) describes the tools and approaches used for the development of the workflow, the results section(Section 3) presents and compares the obtained results from the implemented pipeline. Finally, the discussion section(Section 4) interprets the findings in the context of the reference workflow and discusses potential improvements and future extensions.

2 Methodology

2.1 Nf-core pipeline Structure

Nf-core pipelines follow a standardized layout of folders and files, that is generated by the nf-core template command. The most important files and folders are: 1) main.nf: serves as entry point into the pipeline, it is executed when the pipeline is run. 2) nextflow.config file: This file contains the main configurations, such as default pipeline parameters, nextflow configuration options, and metadata such as pipeline version and minimum version of Nextflow. 3) modules: this folder is pipeline specific and contains shared nf-core modules. 4) workflows: contains the main pipeline workflow, which is executed from main.nf. 5) subworkflows: contains smaller local and shared nf-core workflows. 6) conf: contains pipeline configuration files, such as: base.config (CPU/ memory requirements for low-, medium- and high-resource processes), igenomes.config (reference genome locations), and test configuration files for test profiles(Section 3.2). 7) bin: folder contains scripts that can be called directly from Nextflow processes. 8) README.md: every repository that follows the FAIR principles should contain a well documented README.md file, that details how to use the pipeline. 9) CITATIONS.md: lists the tools used by the pipeline, which is important for correct academic work. 10) CHANGELOG.md: documents the changes between releases. This was of lesser importance for this project, as there is only one version. 11) nextflow_schema.json: machine-readable specification of the pipeline parameters. Less important but still part of the standard nf-core structure are: 12) assets: contains static files like sample data sheet templates and email templates.13) docs: contains documentation on usage and output. (nf-core, n.d.-a)

2.2 Implementation

The initial pipeline was implemented using the nf-core template creation tool(nf-core version 4.1.0) (Ewels et al., 2020). The pipeline was implemented with Nextflow (version 26.04.6) (Di Tommaso et al., 2017). The nf-schema plugin was specified to version 2.7.2, due to the attempt to enable correct parameter interpretation from the command line.
The pipeline was executed using Docker (version 29.9.1) (Merkel, 2014) within a miniconda environment (version 26.1.1) (Anaconda, 2016), that was used to install nextflow and nf-core tools in an isolated environment. The machine used, was a laptop with Ubuntu OS (version 24.04.5 LTS), which was equipped with an Intel Core Ultra 7 155H processor (16 physical cores, 22 threads) and 30 GiB total RAM available on Linux. The source code is available on github (https://github.com/NinaBee13/Computational_Workflows_project_RoehreSchuele). The pipeline was run with the commands:

nextflow run Computational_Workflows_project_RoehreSchuele
-profile docker
–input samplesheet.csv
–fasta reference.fa
–gtf annotation.gtf
–outdir results
–concat_duplicate_samples boolean

nextflow run Computational_Workflows_project_RoehreSchuele
-profile test,docker
–concat_duplicate_samples boolean
–outdir test_results

The pipeline was assembled from standardized nf-core modules and subworkflows (Table 3 & Table 4). The following nf-core modules were used: cat, fastqc, fastp, gunzip, hisat2, multiqc, picard, rsem, salmon, samtools, tximeta untar, as well as the following subworkflows: bam_sort_stats_samtools, bam_stats_samtools, fastq_align_hisat2 (Table 4). For exact software versions of each tool used within the modules and subworkflows can be found in Table 3. They are also reported in the "pipeline_info/" folder of the pipeline output.

3 Results

3.1 Pipeline implementation and execution

The developed Nextflow pipeline integrates the following RNA-seq processing steps: quality control, adapter and quality trimming, read alignment, marking duplicate reads, and the production of a transcripts per million (TPM) table by genes and samples. After preprocessing, the workflow separates into two analysis branches: a genome alignment branch using HISAT2 and a quantification branch using Salmon (Figure 1).

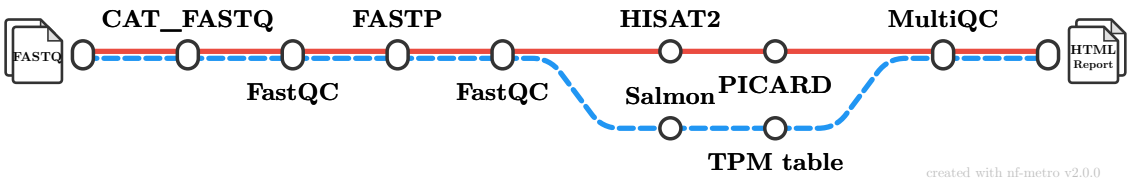


Figure 1: Metromap of the pipeline picturing main steps. Red branch: Alignment with HISAT2. Blue dashed branch: Pseudoalignment and Quantification with Salmon

3.2 Test sets and profiles

To validate the functionality of the pipeline two test profiles were implemented for the pipeline. Both can be found within the pipeline repository under the "conf/" folder. Both test profiles use testdata from nf-core ((nf-core, n.d.-b)) as samplesheet(Table 1 & Table 2), fasta and gtf as input, using established file formats supports interoperability and reusability.
When executing a test-profile the pipeline first checks the sample sheet for non-unique sample names, if there are non-unique sample names present it terminates with an error message. The user can define if the data of non-unique sample should be concatenated by the module CAT_FASTQ by setting the flag "–concat_duplicate_samples true" to avoid the termination. An additional parameter validation check using an if-statement was implemented because boolean values were still not interpreted correctly despite the explicitly specified nf-schma plugin version.

- test profile 1(test.config): with non-unique sample names from github (nf-core, n.d.-b). Was used to test the added –concat_duplicate_samples flag.
- test profile 2(test_without_duplicates.config): modified samplesheet, with unique sample names from github (nf-core, n.d.-b) without duplicated samples (sample sheet in test/ folder).

Both test profiles were executed successfully, demonstrating that the pipeline supports input sample sheets with and without non-unique sample names.

Sample	**FASTQ 1**	**FASTQ 2**	**Strandedness**
WT_REP1	SRR6357070_1.fastq.gz	SRR6357070_2.fastq.gz	auto
WT_REP1	SRR6357071_1.fastq.gz	SRR6357071_2.fastq.gz	auto
WT_REP2	SRR6357072_1.fastq.gz	SRR6357072_2.fastq.gz	reverse
RAP1_UNINDUCED_REP1	SRR6357073_1.fastq.gz	—	reverse
RAP1_UNINDUCED_REP2	SRR6357074_1.fastq.gz	—	reverse
RAP1_UNINDUCED_REP2	SRR6357075_1.fastq.gz	—	reverse
RAP1_IAA_30M_REP1	SRR6357076_1.fastq.gz	SRR6357076_2.fastq.gz	reverse

Table 1: Sample sheet for test profile “test” including duplicate sample names and single and paired-end sequencing data.

Sample	**FASTQ 1**	**FASTQ 2**	**Strandedness**
WT_REP1	SRR6357070_1.fastq.gz	SRR6357070_2.fastq.gz	auto
WT_REP2	SRR6357072_1.fastq.gz	SRR6357072_2.fastq.gz	reverse
RAP1_UNINDUCED_REP1	SRR6357073_1.fastq.gz	—	reverse
RAP1_UNINDUCED_REP2	SRR6357075_1.fastq.gz	—	reverse
RAP1_IAA_30M_REP1	SRR6357076_1.fastq.gz	SRR6357076_2.fastq.gz	reverse

Table 2: Sample sheet for test profile “test_without_duplicates” including single and paired-end sequencing data.

3.3 Processing steps and generated outputs

The raw reads defined in the sample sheet (Table 1 and Table 2) are first processed by the nf-core FASTQC module for quality control and the nf-core Fastp module for adapter and quality trimming . After trimming a second quality control step on the trimmed data is performed with FastQC. Following preprocessing the workflow separates into two processing branches:
For genome alignment, the reference FASTA and GTF annotation files are decompressed using GUNZIP if necessary. To prepare the required input files for the HISAT2 Module, HISAT2_EXTRACTSPLICESITES was used to extract known splice sites from the GTF file, HISAT2_BUILD generated the HISAT2 genome index and SAMTOOLS_FAIDX created a FASTA index. The prepared reference files and trimmed reads were subsequently passed to the fastq_align_hisat2 subworkflow, which performs read alignment and downstream BAM sorting, indexing and statistical analysis in the subworkflows bam_sort_stats_samtools and bam_stats_samtools.
The resulting sorted BAM files, together with the reference genome FASTA file and its index generated by SAMTOOLS_FAIDX, were subsequently processed by the Picard module to mark

duplicate reads.
In the quantification branch the pipeline generates a transcript fasta file using reference genome FASTA and the GTF file with RSEM. SALMON_INDEX builds a Salmon index from the transcript fasta file and reference genome. SALMON_QUANT then estimates transcript abundances using the trimmed FASTQ reads and the prepared reference files. SALMON_QUANT is configured to select the appropriate library type based on the provided strandedness information in the meta data and whether the reads are single-end or paired-end. For paired-end samples the pipeline assumes an inward-facing library orientation. If no strandedness information is available or the strandedness is set to "auto" in the metadata, Salmon infers the library type for this sample. This allows Salmon to consider the orientation of the reads and potentially achieve more accurate transcript abundance estimates.
To map the estimated transcript abundances to their corresponding genes the module CUSTOM_TX2GENE uses the GTF annotation and Salmon output. The mapping, together with the Salmon quantification results from all samples, is then summarized by TXIMETA_TXIMPORT to produce the TPM table by genes and by samples Finally, the nf-core MultiQC module summarizes quality control steps and TPM table results into a single HTML report.

3.4 Workflow performance and parallelization

Running the test with duplicate sample names results in an execution time of 1m 12s, while the test profile without duplicate sample names had an execution time of 1m 03s. Both runs used 0.1 CPU hours and completed 69 tasks successfully.
Nextflow enables parallel execution of processes as soon as their required inputs become available. In our pipeline parallelization is possible. The initial quality control with FastQC and read trimming with fastp can be performed in parallel. Furthermore, the HISAT2 alignment and Salmon quantification branches can run independently after trimming. Inspecting both execution_timeline HTML files confirms that parallel execution of tasks was performed to some extend (Figure 2 and Figure 3).

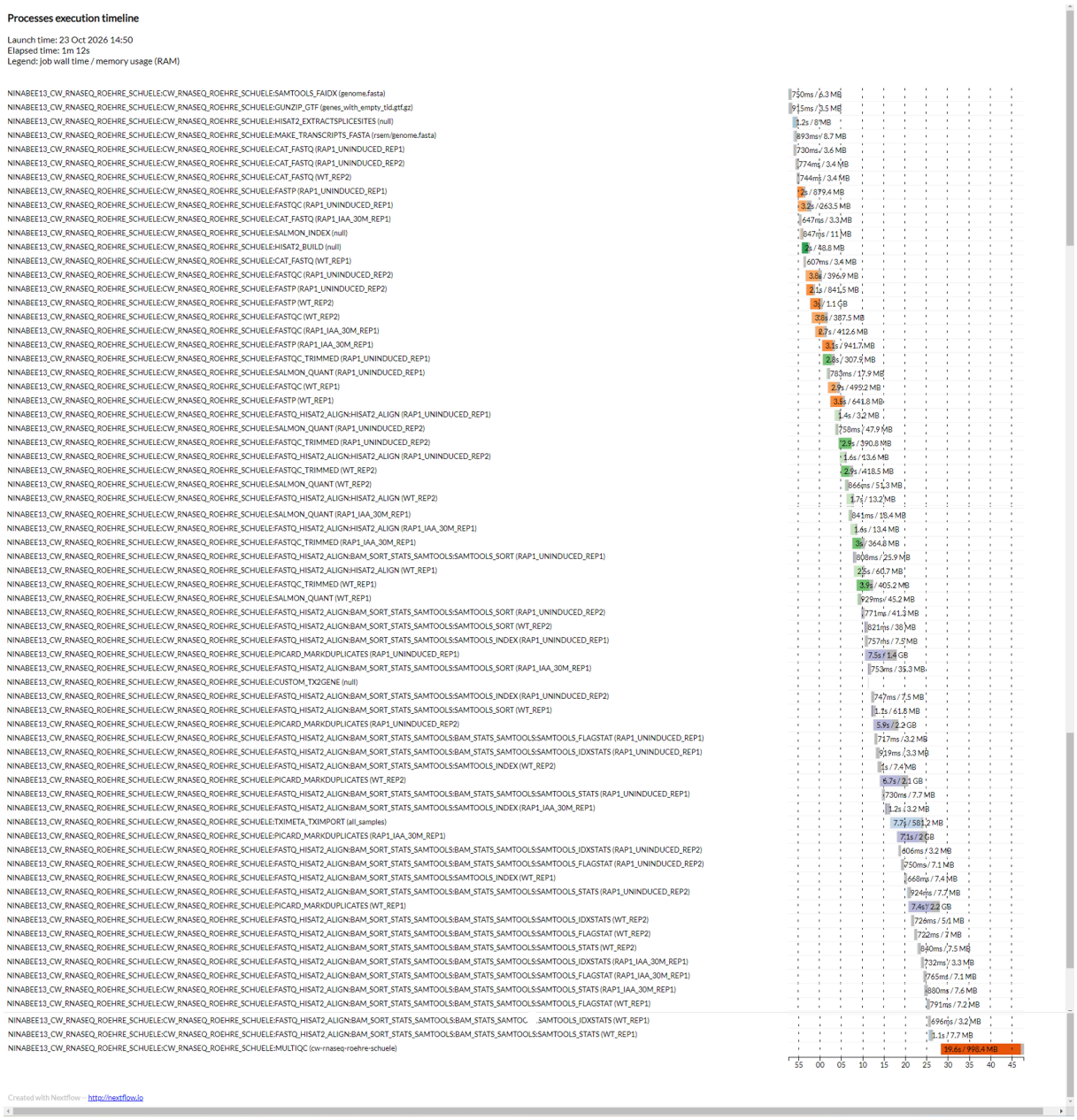


Figure 2: Execution timeline generated by Nextflow for the test profile with duplicate sample names.Colours indicate tasks belonging to the same process and the actual execution time of the task. Gray section before the colored section indicates time between task submission and the start of the execution. Gray sections after the colored section indicates time needed to finalize the task after its main execution, for example clean up and other processing overhead.



Figure 3: Execution timeline generated by Nextflow for the test profile without duplicate sample names.Colours indicate tasks belonging to the same process and the actual execution time of the task. Gray section before the colored section indicates time between task submission and the start of the execution. Gray sections after the colored section indicates time needed to finalize the task after its main execution, for example clean up and other processing overhead.

4 Discussion

4.1 Evaluating the pipeline according to FAIR principles

In the following we want to evaluate the pipeline according to the FAIR principles for research software (Barker et al., 2022), summarized in Table 5.

Findable and Accessible: An important aspect improving findability and accessibility is that the pipeline is hosted in a publicly searchable and accessible GitHub repository and made available as a release (F4, F1.2). Users should be able to retrieve the source code through standardized

protocols such as HTTPs, facilitating easy access to the pipeline and its reuse by others (A1, A1.1). To guarantee long-term accessibility of the release we could archive it in a repository such as Zendo, this would guarantee accessibility even if the GitHub repository is set to private in the future and would provide a Digital Object Identifier (DOI) for citation and improved findability (F1, F4, A1, A2, R1.2) (Zenodo, n.d.).

Interoperable: The pipeline uses standard file formats (FASTQ, FASTA, GTF, BAM) which can be further processed by other regularly used bioinformatics software (I1) (Li et al., 2009). The use of standard file and samplesheet formats and standardized nf-core modules and Nextflow supports interoperability by allowing individual processing steps to be integrated into other computational workflows (I1, R3) (Ewels et al., 2020).

Reusability: The use of standardized modules and subworkflows also makes the pipeline easier to reuse, adapt for different analyses and improve reproducibility (R3) (Ewels et al., 2020). Further the provided README and documentation files aim to describe the relevant attributes of the pipeline for easier reuse by others (R1, F2). The Git commit history documents the history of changes in the pipeline, supporting provenance (R1.2). Another important aspect to make a pipeline reusable is the provided MIT license, which is also regularly used for nf-core modules (R1.1) (choosealicense, n.d.). Without a license other researchers do not have the permission to reuse, modify or redistribute the pipeline to reproduce results (GITHUB, n.d.).

Compared with conventional bash scripting, Nextflow also facilitates reproducibility by managing process dependencies and supporting containerized software environments (Di Tommaso et al., 2017; Ewels et al., 2020). Furthermore, the use of Docker containers makes the reuse easier by providing the required software environments. This avoids the need for users to install each tool individually (Merkel, 2014).

4.2 Outlook for future improvements

One future improvement is to add alternative modules to the current implemented ones to make the pipeline more flexible and give the user the opportunity to choose the tool of their personal preference. For example we could add Trim Galore as an alternative to fastp for adapter and quality trimming.

Our pipeline already covers preprocessing and quantification steps and generates a gene-level TPM table. For further analysis of differentially expressed genes between conditions we could add the module deseq2_differential. This would also allow us to perform a sample similarity analysis using PCA, which could help identify potential outliers and problems in the sequencing data (Love et al., 2014).

Currently the pipeline determines the Salmon library type based on the strandedness information provided in the metadata and whether the reads are single- or paired-end. For paired-end reads with explicitly specified strandedness, it assumes an inward facing relative orientation (I), as commonly used in sequencing libraries created with standard Illumina protocols, like our test datasets. (Wu et al., 2018). To make the pipeline suitable for a wider range of library preparation techniques, the support of different paired-end orientations,such as outward orientation (O), could be introduced (Salmon Development Team, n.d.). For example the nf-core rnaseq pipeline allows users to set the library type, and therefore different relative read orientation, explicitly with the flag “–salmon_quant_libtype” (nf-core, n.d.-c).

Additionally, we could include Unique molecular identifier support, this would further increase the accuracy of the quantification, since UMIs allow us to identify PCR duplicates (Smith et al., 2017). This could be done by integrating the required tools and by using another flag option, similar to the "–with_umi " option provided by the nf-core/rnaseq pipeline (nf-core, n.d.-c). However, this functionality requires that UMIs were incorporated into the sequencing library during library preparation, before PCR amplification.

Although accessibility for large language models (LLMs) is not an explicit FAIR requirement, evaluating how easily such tools can access the pipeline and its metadata could be an interesting direction considering the frequent and increasing use of such tools in scientific research (Chugunova et al., 2026).

Additionally, further compartmentalization into subworkflows, even for smaller modules could make debugging easier as the pipeline grows.

4.3 Conclusion

In this project we successfully implemented a basic RNA-seq pipeline using Nextflow, standardized nf-core modules and nf-core subworkflows, which support reusability and reproducibility. The pipeline includes quality control steps, adapter and quality trimming, genome alignment and quantification to generate a TPM table by genes and samples and provides the summarized results in a HTML report. The pipeline was successfully tested using two test profiles, with and without duplicate sample names.
The use of Nextflow, nf-core modules and Docker supports the FAIR principlee. Particularly interoparbility and reusability are supported through standardized workflows, the used file formats and reproducible software enviroments.
Findability, accessibility and reusability are supported through the publicly available GitHub repository, documentation and the provided MIT licence. To further improve compliance with the FAIR principles, the pipeline could be archived in a repository such as Zenodo, providing a DOI and searchable metadata.

References

Anaconda. (2016, November). *Anaconda Software Distribution*. Anaconda, Inc. https://anaconda.com/

Andrews, S. (2010,). *FastQC: A Quality Control Tool for High Throughput Sequence Data*.

Barker, M., Chue Hong, N. P., Katz, D. S., Lamprecht, A.-L., Martinez-Ortiz, C., Psomopoulos, F., Harrow, J., Castro, L. J., Gruenpeter, M., Martinez, P. A., & Honeyman, T. (2022). Introducing the FAIR Priciples for research software. *Sci Data*, *9*, 622. https://doi.org/10.1038/s41597-022-01710-x

Chen, S., Zhou, Y., Chen, Y., & Gu, J. (2018). fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics*, *34*(17), i884–i890. https://doi.org/10.1093/bioinformatics/bty560

choosealicense. (n.d.). *MIT License*. Retrieved October 23, 2026, from https://choosealicense.com/licenses/mit/

Chugunova, M., Harhoff, D., Hölzle, K., Kaschub, V., Malagimani, S., Morgalla, U., & Rose, R. (2026). Who uses AI in research, and for what? Large-scale survey evidence from Germany. *Research Policy*. https://doi.org/10.1016/j.respol.2025.105381

Di Tommaso, P., Chatzou, M., Floden, E. W., Barja, P. P., Palumbo, E., & Notredame, C. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, *35*(4), 316–319. https://doi.org/10.1038/nbt.3820

Dobin, A., Davis, C. A., Schlesinger, F., Drenkow, J., Zaleski, C., Jha, S., Batut, P., Chaisson, M., & Gingeras, T. R. (2013). STAR: ultrafast universal RNA-seq aligner. *Bioinformatics*, *29*(1), 15–21. https://doi.org/10.1093/bioinformatics/bts635

Ewels, P. A., Peltzer, A., Fillinger, S., Patel, H., Alneberg, J., Wilm, A., Garcia, M. U., Di Tommaso, P., & Nahnsen, S. (2020). The nf-core framework for community-curated bioinformatics pipelines. *Nature Biotechnology*, *38*(3), 276–278. https://doi.org/10.1038/s41587-020-0439-x

Free Software Foundation. (n.d.-a). *GNU Coreutils*. Retrieved https://www.gnu.org/software/coreutils/

Free Software Foundation. (n.d.-b). *GNU Gzip*. Retrieved https://www.gnu.org/software/gzip/

GITHUB. (n.d.). *Licensing a repository*. Retrieved October 23, 2026, from https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/licensing-a-repository

Kim, D., Paggi, J. M., Park, C., Bennett, C., & Salzberg, S. L. (2019). Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype. *Nature Biotechnology*, *37*, 907–915. https://doi.org/10.1038/s41587-019-0201-4

Koch, C. M., Chiu, S. F., Akbarpour, M., Bharat, A., Ridge, K. M., Bartom, E. T., & Winter, D. R. (2018). A Beginner's Guide to Analysis of RNA Sequencing Data. *American Journal of Respiratory Cell and Molecular Biology*, *59*(2), 145–157. https://doi.org/10.1165/rcmb.2017-0430TR

Leipzig, J. (2017). A review of bioinformatic pipeline frameworks. *Briefings in Bioinformatics*, *18*(3), 530–536. https://doi.org/10.1093/bib/bbw020

Li, B., & Dewey, C. N. (2011). RSEM: accurate transcript quantification from RNA-Seq data with or without a reference genome. *BMC Bioinformatics*, *12*(1), 323. https://doi.org/10.1186/1471-2105-12-323

Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., & Genome Project Data Processing Subgroup, 1 000. (2009). The Sequence Alignment/Map format and SAMtools. *Bioinformatics*, *25*(16), 2078–2079. https://doi.org/10.1093/bioinformatics/btp352

Love, M. I., Huber, W., & Anders, S. (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology*, *15*, 550. https://doi.org/10.1186/s13059-014-0550-8

Love, M. I., Soneson, C., Hickey, P. F., Johnson, L. K., Pierce, N. T., Shepherd, L., Morgan, M., & Patro, R. (2020). Tximeta: Reference sequence checksums for provenance identification in RNA-seq. *Plos Computational Biology*, *16*(2), e1007664. https://doi.org/10.1371/journal.pcbi.1007664

Merkel, D. (2014). Docker: lightweight Linux containers for consistent development and deployment. *Linux J.*, *2014*(239).

nf-core. (n.d.-c). *nf-core/rnaseq: Usage*. Retrieved October 23, 2026, from https://nf-co.re/rnaseq/usage

nf-core. (n.d.-a). *Pipeline file structure*. Retrieved October 21, 2026, from https://nf-co.re/docs/developing/pipelines/template-files

nf-core. (n.d.-b). *samplesheet_test.csv*. Retrieved October 21, 2026, from https://github.com/nf-core/test-datasets/blob/rnaseq/samplesheet/v3.10/samplesheet_test.csv

nf-core/rnaseq. (n.d.). *nf-core/rnaseq*. Zenodo. https://doi.org/10.5281/zenodo.1400710

Patro, R., Duggal, G., Love, M. I., Irizarry, R. A., & Kingsford, C. (2017). Salmon provides fast and bias-aware quantification of transcript expression. *Nature Methods*, *14*, 417–419. https://doi.org/10.1038/nmeth.4197

Picard toolkit. (2019,). Broad Institute.

Rossum, G. van, & Boer, J. de. (1991). Interactively Testing Remote Servers Using the Python Programming Language. *CWI Quarterly*, *4*(4), 283–303. https://ir.cwi.nl/pub/18204

Salmon Development Team. (n.d.). *Fragment Library Types*. Retrieved October 23, 2026, from https://salmon.readthedocs.io/en/stable/library_type.html

Sansone, S.-A., McQuilton, P., Rocca-Serra, P., Gonzalez-Beltran, A., Izzo, M., Lister, A. L., Thurston, M., & the FAIRsharing Community. (2019). FAIRsharing as a community approach to standards, repositories and policies. *Nature Biotechnology*, *37*(4), 358–367. https://doi.org/10.1038/s41587-019-0080-8

Smith, T., Heger, A., & Sudbery, I. (2017). UMI-tools: modeling sequencing errors in Unique Molecular Identifiers to improve quantification accuracy. *Genome Research*, *27*(3), 491–499. https://doi.org/10.1101/gr.209601.116

Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., & others. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, *3*, 160018. https://doi.org/10.1038/sdata.2016.18

Wu, A. C. K., Patel, H., Chia, M., Moretto, F., Frith, D., Snijders, A. P., & Werven, F. J. van. (2018). Repression of Divergent Noncoding Transcription by a Sequence-Specific Transcription Factor. *Molecular Cell*, *72*(6), 942–954.e7. https://doi.org/10.1016/j.molcel.2018.10.018

Zenodo. (n.d.). *Archive Software*. Retrieved October 24, 2026, from https://help.zenodo.org/docs/github/archive-software/

A Appendix

Workflow component	Software	Version	Software Citation
CAT_FASTQ	cat	9.5	(Free Software Foundation, n.d.-a)
CUSTOM_TX2GENE	Python	3.10.4	(Rossum & Boer, 1991)
FASTP	fastp	1.3.6	(Chen et al., 2018)
FASTQC	FastQC	0.12.1	(Andrews, 2010)
FASTQC_TRIMMED	FastQC	0.12.1	(Andrews, 2010)
GUNZIP_GTF	gunzip	1.13	(Free Software Foundation, n.d.-b)
HISAT2_BUILD	HISAT2	2.2.3	(Kim et al., 2019)
HISAT2_ EXTRACTSPLICESITES	HISAT2; SAMtools	2.2.3; 1.24	(Kim et al., 2019), (Li et al., 2009)
HISAT2_ALIGN	HISAT2; SAMtools	2.2.3; 1.24	(Kim et al., 2019), (Li et al., 2009)
MAKE_TRANSCRIPTS_ FASTA	RSEM; STAR	1.3.1; 2.7.10a	(Li & Dewey, 2011), (Dobin et al., 2013)
PICARD_ MARKDUPLICATES	Picard	3.5.0	(*Picard Toolkit*, 2019)
SALMON_INDEX	Salmon	2.7.0	(Patro et al., 2017)
SALMON_QUANT	Salmon	2.7.0	(Patro et al., 2017)
SAMTOOLS_FAIDX	SAMtools	1.24	(Li et al., 2009)
SAMTOOLS_FLAGSTAT	SAMtools	1.24	(Li et al., 2009)
SAMTOOLS_IDXSTATS	SAMtools	1.24	(Li et al., 2009)
SAMTOOLS_INDEX	SAMtools	1.24	(Li et al., 2009)
SAMTOOLS_SORT	SAMtools	1.24	(Li et al., 2009)
SAMTOOLS_STATS	SAMtools	1.24	(Li et al., 2009)
TXIMETA_TXIMPORT	tximeta	1.20.1	(Love et al., 2020)

Table 3: Workflow components and their software versions. Note that MAKE_TRANSCRIPTS_FASTA reports STAR version, even when STAR is not used.

Type	Component	Function in the analysis
Module	`cat (cat_fastq)`	Concatenation of fastq files
Module	`fastqc`	Quality assessment of sequencing reads with FastQC
Module	`fastp`	Adapter removal and quality trimming on reads
Module	`gunzip`	Decompression of compressed files and compression of files
Module	`hisat2 (hisat2_align, hisat2_build, hisat2_extractsplicesites)`	Alignment of sequencing reads to the reference genome
Module	`multiqc`	Aggregation and visualization of quality-control results and analyses
Module	`picard (picard_markduplicates)`	Identification and marking of duplicate reads
Module	`rsem (rsem_prepare-reference)`	Prepares a reference genome for RNA-Seq by Expectation-Maximization (RSEM)
Module	`salmon (salmon_index, salmon_quant)`	Transcript quantification
Module	`samtools(samtools_faidx, samtools_flagstat, samtools_idxstats, samtools_index, samtools_sort, samtools_stats)`	Sorting, indexing, and statistical analysis of alignment files
Module	`tximeta (tximeta_tximport)`	Import of transcript abundances and estimated counts
Module	`untar`	Extraction of files from tar archives
Subworkflow	`bam_sort_stats_samtools`	Sorting of SAM, BAM and CRAM files
Subworkflow	`bam_stats_samtools`	Generation of alignment statistics using SAMtools
Subworkflow	`fastq_align_hisat2`	Alignment of sequencing reads with HISAT2

Table 4: nf-core modules and subworkflows used in the RNA-seq analysis and their respective functions.

Principle	Description
F: Software, and its associated metadata, is easy for both humans and machines to find.	
F1	Software is assigned a globally unique and persistent identifier.
F1.1	Components of the software representing levels of granularity are assigned distinct identifiers.
F1.2	. Different versions of the software are assigned distinct identifiers.
F2	Software is described with rich metadata.
F3	Metadata clearly and explicitly include the identifier of the software they describe.
F4	Metadata are FAIR, searchable and indexable.
A: Software, and its metadata, is retrievable via standardised protocols.	
A1	Software is retrievable by its identifier using a standardised communications protocol.
A1.1	The protocol is open, free, and universally implementable.
A1.2	The protocol allows for an authentication and authorization procedure, where necessary.
A2	Metadata are accessible, even when the software is no longer available.
I: Software interoperates with other software by exchanging data and/or metadata, and/or through interaction via application programming interfaces (APIs), described through standards.	
I1	Software reads, writes and exchanges data in a way that meets domain-relevant community standards.
I2	Software includes qualified references to other objects.
R: Software is both usable (can be executed) and reusable (can be understood, modified, built upon, or incorporated into other software).	
R1	Software is described with a plurality of accurate and relevant attributes.
R1.1	Software is given a clear and accessible license.
R1.2	Software is associated with detailed provenance.
R2	Software includes qualified references to other software.
R3	Software meets domain-relevant community standards.

Table 5: FAIR Principles for Research Software (FAIR4RS). Defined by Barker et al. (2022) (Barker et al., 2022).